

Министерство образования и науки Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ
ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ

Факультет программной инженерии и компьютерных технологий

Отчёт по лабораторной работе №1

Выполнил: Семикозов Ян Леонидович

Группа: Р4119

Преподаватель: Табунщик Сергей Михайлович

1 Цель работы

Получить общее представление о системе команд RISC-V путём решения несложной прикладной задачи – найти наименьший элемент на диагонали в матрице 6×6 .

2 Ход работы

В данной работе используется архитектура RISC-V с набором команд RV32I. В соответствии с *соглашением о вызовах функций RISC-V* аргументы функции передаются через регистры **a0-a7**, а результат возвращается в регистрах **a0**, **a1**. Ниже приведена сводная таблица используемых регистров и их назначение в программе (таблица 1).

Таблица 1. Назначение используемых регистров

Регистры	Номер (x#)	Назначение в коде
ra	x1	Адрес возврата из функции
sp	x2	Указатель на вершину стека
s0	x8	Адрес матрицы
s1	x9	Текущее минимальное значение
s2	x10	Индекс цикла <i>i</i>
t0	x5	Граница цикла
t1	x6	Смещение в матрице
t2	x7	Текущий элемент в матрице
a0	x10	Аргумент функции/код системного вызова, возвращаемое значение
a1	x11	Аргумент для <code>ecall</code>

Прежде чем описывать функциональную часть программы разместим входные и неизменяемые данные в секции `.data`.

```
1      .data
2  matrix:
3      .word 12, 7, 31, 4, 5, 6
4      .word 8, 10, 2, 11, 1, 9
5      .word 14, 15, 6, 13, 12, 7
6      .word 16, 17, 18, 5, 19, 20
7      .word 21, 22, 23, 24, 9, 25
8      .word 26, 27, 28, 29, 30, 10
9
10     minelemstr:
11     .asciiiz "min element: "
```

Листинг 1. Код программы на языке ассемблера RISC-V

Все инструкции программы разместим в секции `.text`; для глобальной видимости функций используется метка `.globl`. В функции *findMinElement* вначале создадим стековый фрейм для сохранения регистров, которые будут изменяться. При этом выделим 16 байт на стек и сохраним в него регистры `ra`, `s0`, `s1` и `s2`. При завершении функции восстановим сохранённые значения регистров и освободим стековый фрейм. Ниже в листинге 2 приведён полный код функции *findMinElement*:

```

1      .text
2      .globl findMinElement
3      findMinElement:
4          addi sp, sp, -16
5          sw ra, 12(sp)
6          sw s0, 8(sp)
7          sw s1, 4(sp)
8          sw s2, 0(sp)
9          mv s0, a0
10         lw s1, 0(s0)
11         li s2, 1
12
13     loop:
14         li t0, 6
15         bge s2, t0, endloop
16         li t1, 7
17         mul t1, t1, s2
18         slli t1, t1, 2
19         add t1, s0, t1
20         lw t2, 0(t1)
21         blt t2, s1, updateMin
22         j next_i
23
24     updateMin:
25         mv s1, t2
26
27     next_i:
28         addi s2, s2, 1
29         j loop
30
31     endloop:
32         mv a0, s1
33         lw ra, 12(sp)
34         lw s0, 8(sp)
35         lw s1, 4(sp)
36         lw s2, 0(sp)
37         addi sp, sp, 16
38         ret

```

Листинг 2. Код функции *findMinElement* на языке ассемблера RISC-V

Ниже в листинге 3 приведён полный код функции *main*, которая вызывает функцию поиска минимального элемента и выводит результат на экран с использованием системных вызовов. Код системного вызова записывается в регистр **a7**, а аргумент в **a1**:

- Код 1 – вывод целого числа.

- Код 4 – вывод строки.
- Код 11 – вывод символа.

```
1  main:
2      addi sp, sp, -16
3      sw ra, 12(sp)
4
5      la a0, matrix
6      call findMinElement
7
8      mv t0, a0
9
10     # print string
11     la a1, minelemstr
12     li a0, 4
13     ecall
14
15     # print integer
16     mv a1, t0
17     li a0, 1
18     ecall
19
20     # print newline
21     li a1, 10
22     li a0, 11
23     ecall
24
25
26     lw ra, 12(sp)
27     addi sp, sp, 16
28     li a0, 0
29     ret
30
```

Листинг 3. Код функции *main* на языке ассемблера RISC-V

Ниже в листинге 4 приведён полный код программы на языке C, решающей ту же задачу.

```

1  #include <stdio.h>
2
3  #define N 6
4  #define M 6
5
6  int findMinElement(int matrix[N][M])
7  {
8      int min = matrix[0][0];
9
10     for (int i = 1; i < N && i < M; i++) {
11         if (matrix[i][i] < min) {
12             min = matrix[i][i];
13         }
14     }
15
16     return min;
17 }
18
19 int main()
20 {
21     int matrix[N][M] = {
22         { 12, 7, 31, 4, 5, 6 },
23         { 8, 10, 2, 11, 1, 9 },
24         { 14, 15, 6, 13, 12, 7 },
25         { 16, 17, 18, 5, 19, 20 },
26         { 21, 22, 23, 24, 3, 25 },
27         { 26, 27, 28, 29, 30, 4 }
28     };
29
30     int min = findMinElement(matrix);
31
32     printf("min element: %d\n", min);
33
34     return 0;
35 }

```

Листинг 4. Код программы на C

Сборка и запуск программы осуществлялись с использованием собранного кросс-компилятора *RISC-V* и эмулятора *QEMU*. Ниже приведены команды, использованные для сборки и запуска программы из корня проекта:

```
make -C src && qemu-riscv32 src/main.elf
```

Переписанный код на языке ассемблера был проверен в симуляторе *Venus*.

3 Выводы

В ходе выполнения лабораторной работы изучены базовые принципы написания кода под архитектуру RISC-V. Реализована программа на языке С и её эквивалент на языке ассемблера. Ознакомились с процессом сборки и запуска программ под архитектуру RISC-V.